

Using Retries to Handle Failures in Microservices

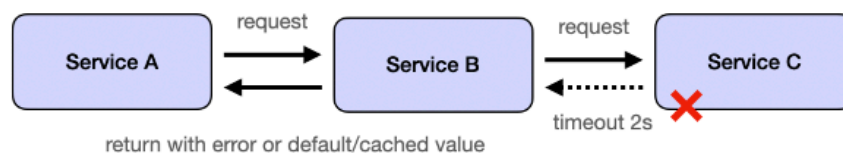
 systemdesignschool.io/fundamentals/retries



Retries

Timeouts prevent threads from blocking forever by aborting slow requests. But some failures are transient. A network packet gets dropped. A service restarts. A database connection pool is momentarily full. These failures resolve quickly. Retrying the same request often succeeds.

When Retries Help



A mobile app calls the API gateway. The gateway forwards the request to the user service. A network switch hiccups and drops a packet. The request fails. The gateway immediately retries. The network is fine now. The retry succeeds. The user never knows anything failed.

Without retries, the user sees an error. They manually retry by tapping the button again. The second tap works. Automatic retries provide a better experience by hiding transient failures.

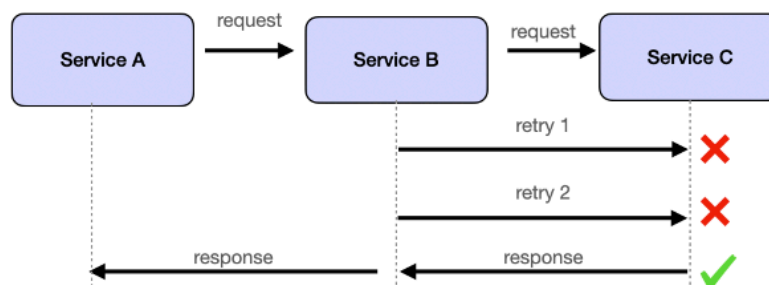
When Not to Retry

Retries only help with transient failures. If a user requests a resource that does not exist, retrying returns the same 404 error. If authentication fails because credentials are wrong, retrying with the same credentials fails again.

Check the error type before retrying. Network errors and 5xx server errors might be transient. Retry them. 4xx client errors like 400 Bad Request or 401 Unauthorized are not transient. Do not retry them.

Non-idempotent operations need care. Creating a resource might succeed on the server but the response gets lost. Retrying creates a duplicate. Use idempotency keys. The client sends a unique ID with each request. The server stores successful operations by ID. A retry with the same ID returns the original result without creating a duplicate.

Exponential Backoff



Immediate retries can overwhelm a struggling service. If the service is overloaded, retrying instantly adds more load. The service falls further behind.

Exponential backoff spaces out retries. Wait 1 second before the first retry. If that fails, wait 2 seconds. Then 4 seconds. Then 8 seconds. This gives the service time to recover.

```
import time
import requests

def call_service_with_retry(url, max_retries=3):
    for attempt in range(max_retries):
        try:
            response = requests.get(url, timeout=2)
            return response.json()
        except requests.exceptions.RequestException as e:
            if attempt == max_retries - 1:
                raise
            wait_time = 2 ** attempt # 1, 2, 4 seconds
            time.sleep(wait_time)
```

Adding Jitter

Exponential backoff alone has a problem. If 1000 requests fail simultaneously, they all retry after 1 second. Then all retry after 2 seconds. This synchronized retry storm can overwhelm the service when it starts recovering.

Jitter adds randomness to wait times. Instead of waiting exactly 2 seconds, wait between 1 and 2 seconds. Each client retries at a slightly different time. The load spreads out.

```
import random wait_time = 2 ** attempt jittered_wait = wait_time * (0.5 +
random.random() * 0.5) time.sleep(jittered_wait)
```

Retry Limits

Always set a maximum number of retries. Three attempts is common. After three failures, give up. Return an error or use a fallback. Retrying forever wastes resources and delays error handling.

Track total elapsed time too. If the original request plus retries takes 10 seconds, stop even if you have retries left. The user has probably given up or the operation is no longer relevant.

Combining with Circuit Breakers

Retries work well for occasional failures. But if Service B is completely down, retrying wastes time. Every request tries three times, waits for exponential backoff, then fails. This adds latency without helping.

Circuit breakers detect persistent failures and skip retries. After 10 consecutive failures, stop calling Service B entirely. Return errors immediately or use a fallback. This is covered in detail in the [circuit breaker](#) section.